

Coding Sample: Escaping Inequality in India (Selected Excerpts)

Rishav Roy

What follows are excerpts from the 138-page code appendix to my independent paper and writing sample, “Escaping Inequality in India: The Role of English-Medium Instruction.”

The next 25 pages illustrate data ingestion and harmonization, complex survey econometrics, IV estimation, and careful diagnostics. Each excerpt is a part of the code chunks listed below:

14-16: Scalable **import, cleaning, and merging** of multi-file census and geospatial data.

- Chunk 4: Import key data files

31-32: **QA and identifier-disambiguation checks** for district-level crosswalk.

- Chunk 6: Match districts: Construct district tracking dfs

50-52: **Testing for systematic missingness** using the Benjamini–Hochberg procedure and parallelized logistic regressions.

- Chunk 8: Participation model: Are NAs randomly distributed

55-57: **Survey-weighted probit estimation**, average marginal effects derivation, and setup for sample selection correction under a complex survey design.

- Chunk 9: Participation model: Estimation and IMR
- Chunk 10: Participation model: Derive AME

80-88: Reusable **record-linkage functions** using cascading fuzzy matches and unmatched/false-match diagnostics to harmonize district identifiers across years and data sources.

- Chunk 16: Match districts: Test joining methods
- Chunk 17: Match districts: Define joining functions

105-107: Reusable **IV specification and estimation pipeline** with clustered inference, plus contiguity-based spatial weights and setup for spatially lagged models.

- Chunk 24: Geospatial: Neighbor list construction
- Chunk 25: 2SLS: Controls and 2SLS formula
- Chunk 26: 2SLS: Baseline 2SLS models

111-112: **Robustness checks and diagnostics** for multicollinearity, spatial autocorrelation.

- Chunk 28: Multicollinearity check
- Chunk 29: Geospatial: Spatial autocorrelation tests

```

#### From the Census of India 2001
data_folder <- "Indian Census 2001"

# start.time <- Sys.time()
# 35 files will be read in. Preallocate list to store their cleaned data
files <- list.files(
  data_folder,
  pattern = "^PC01_C16_\\d{2}\\\\.xls$",
  full.names = TRUE
)
result_list <- vector("list", length(files))

for (i in seq_along(files)) {
  file_path <- files[i]
  temp <- read_excel(file_path, skip = 6, col_names = FALSE)

  # Explicitly set column names
  colnames(temp) <- c("table", "state_code", "district_code", "tehsil_code", "area_name",
    "mother_tongue_code", "mother_tongue", "spkr_tot", "m_spkr_tot",
    "f_spkr_tot", "spkr_urban", "m_spkr_urban", "f_spkr_urban",
    "spkr_rural", "m_spkr_rural", "f_spkr_rural")

  # Extract the state name:
  # Find the first row in the area_name column that begins with "State - "
  state_val <- temp$area_name[grepl("^State - ", temp$area_name)]
  if(length(state_val) == 0) {
    state <- NA_character_
  } else {
    state <- state_val[1] %>%
      # Remove the "State - " prefix

```

```

    str_remove("^State -\\s*") %>%
    # Remove four-digit suffix code
    str_remove("\\s*\\d{4}$") %>%
    # Remove leading/trailing spaces. Squish internal spaces
    str_squish() %>%
    # Convert to title case
    str_to_title()
  }

# Filter the dataframe for rows where both:
# 1. "mother_tongue_code" ends with "0"
# 2. "area_name" begins with "District - "
temp <- temp %>%
  filter(str_detect(as.character(mother_tongue_code), "0$"),
         str_detect(area_name, "^District - "))

# Create "district" and "state" columns
temp <- temp %>%
  mutate(
    # Create "district" by cleaning "area_name"
    district = area_name %>%
      str_remove("^District -\\s*") %>%
      str_remove("\\s*\\d{4}$") %>%
      str_squish() %>%
      # Remove all trailing non-letters
      str_replace("[^[:alpha:]]+$", ""),

    # Create "state" via extracted state name
    state = state
  )

# Clean "mother_tongue" column

```

```

temp <- temp %>%
  mutate(mother_tongue = mother_tongue %>%
    str_remove("^\\d{1,3}\\s+") %>% # Remove leading 1-3 digits and a space
    str_to_title()) # Title case

# Save cleaned df in list
result_list[[i]] <- temp
}

# Combine cleaned dfs
census01 <- bind_rows(result_list)

# end.time <- Sys.time()
# end.time - start.time
# For us, above order (clean files, then bind) was 1.48 secs faster than alternatives.

# Uncomment below to visually verify results
# census01 %>% select(-c(table, tehsil_code, area_name, mother_tongue_code)) %>%
  ↪ group_by(state) %>% distinct(district) %>% View()

# ---

#### District boundaries and regions ####
# From @meyersIndiaOfficialBoundaries2020 via @bhatiaMergingUpdatedDistrictlevel2020
data_folder <- "District Boundaries 2020"

district_bnds_20 <- st_read(
  file.path(data_folder, "district", "in_district.shp"),
  quiet = TRUE) %>%
  rename(district_20 = dtname,
    state_20 = stname) %>%

```

```

# View number of districts each year with the same name despite being in different states

# 1. Filter rows
# For each suffix, we flag rows where the district (for that suffix) appears with more
  ↳ than one distinct state. Then we keep rows where at least one of the suffix
  ↳ comparisons is TRUE.

# Create a list of logical vectors (one per suffix)
keep_list <- lapply(year_suffixes, function(sfx) {

  # Build the column names for this suffix
  state_col <- paste0("state_", sfx)
  district_col <- paste0("district_", sfx)

  # Group the data by district for this suffix and count distinct states
  temp <- district_tracker %>%
    filter(
      !is.na(.data[[district_col]]),
      !is.na(.data[[state_col]])
    ) %>%
    group_by(district = .data[[district_col]]) %>%
    summarise(
      n_states = n_distinct(.data[[state_col]], na.rm = TRUE),
      .groups = "drop"
    )

  # Districts that occur with more than one unique state
  duplicate_districts <- temp %>%
    filter(n_states > 1) %>%
    pull(district)

  # Return a logical vector: TRUE if the district value in the row is in
  ↳ duplicate_districts

```

```

    district_tracker[[district_col]] %in% duplicate_districts
  })

# Combine the logical vectors by taking the rowwise OR (i.e., TRUE if condition holds for
↪ any suffix)
keep_indicator <- Reduce(`|`, keep_list)

# Filter the original data
same_name_districts <- district_tracker[keep_indicator, ]


# 2. Summarize by district for each suffix
# For each suffix, group the filtered data by the district column and count:
# a. the number of rows (n_rows) sharing that district value, and
# b. the number of distinct state values (n_states) in that group.
# Then keep only groups that have more than one unique state.
temp <- lapply(year_suffixes, function(sfx) {
  state_col <- paste0("state_", sfx)
  district_col <- paste0("district_", sfx)
  same_name_districts %>%
    group_by(district_name = .data[[district_col]]) %>%
    summarise(n_rows = n(),
              n_states = n_distinct(.data[[state_col]]),
              .groups = "drop") %>%
    filter(n_states > 1) %>%
    mutate(year_suffix = sfx)
})

# Combine summaries from all suffixes into one data frame
temp <- bind_rows(temp)

# Make it easy to get summary stats
n_same_name_districts <- temp %>%
  group_by(year_suffix) %>%
  summarize(n = n())

# Between min(n_same_name_districts$n) = 3 and max(n_same_name_districts$n) = 5 districts
↪ with the same name in different states in each year of interest

```

```

)

#### Logistic per missing variable ####

check_missing_logit_parallel <- function(df, miss_vars, covars, method_p = "BH") {
  # RHS of regression
  rhs <- paste(covars, collapse = " + ")

  # To fit the binary outcome is.na(m) for each missing var m
  fit_one <- function(m) {
    f <- as.formula(paste0("is.na(", m, ") ~ ", rhs))
    fit <- glm(f, data = df, family = binomial)
    pseudoR2 <- 1 - fit$deviance / fit$null.deviance # Note: No missingness indicator was
    ↪ constant
    gl <- glance(fit) # Currently unused; retained for future diagnostics

    # Return tidy df of coefficients and attach missing var's name
    broom::tidy(fit) %>% # Namespace made explicit for cluster workers
      mutate(
        missing_var = m,
        pseudoR2 = pseudoR2,
        nobs = stats::nobs(fit)
      )
  }

  # Parallelization
  # Cluster with parLapply() if on Windows
  out_list <- if (.Platform$OS.type == "windows") {
    cl <- parallel::makeCluster(max(1, parallel::detectCores() - 1))
    on.exit(parallel::stopCluster(cl), add = TRUE) # Shut down cluster even if errors
    ↪ occur
  }

```

```

parallel::clusterEvalQ(cl, {
  library(broom)
  library(dplyr)
})
parallel::clusterExport(
  cl,
  varlist = c("df", "rhs", "fit_one"),
  envir = environment()
)
parallel::parLapply(cl, miss_vars, fit_one)
} else { # Fork with mclapply()
  parallel::mclapply(miss_vars, fit_one, mc.cores = max(1, parallel::detectCores() -
↪ 1))
} # Returns list of tidy dfs, one per missing var

out <- bind_rows(out_list) # Stack the dfs
out <- out %>%
  group_by(missing_var) %>%
  mutate(
    p_adj = { # Adjust within each missingness model
      adj <- rep(NA_real_, dplyr::n())
      idx <- term != "(Intercept)" # Exclude intercept
      adj[idx] <- p.adjust(p.value[idx], method = method_p)
      adj
    }
  ) %>%
  ungroup()
out
}

covars <- c("SECTOR", "SEX", "AGE", "HH_SIZE", "RELIGION", "SOCIAL_GROUP", "state_0708")

# Run on missing variables defined for all children

```



```

logit_missing_results_all <- check_missing_logit_parallel(selection_df, miss_vars_all,
  ↪ covars, method_p = "BH") # "BH" = Benjamini-Hochberg control on false discovery rate.
  ↪

# "BY" lets independence or positive dependence assumptions weaken to arbitrary
  ↪ dependence, but has weaker power (i.e., more p-vals above 0.05).

sel_logit_NA_all <- logit_missing_results_all %>%
  group_by(missing_var) %>% # Use name from fit_one()
  summarise(
    n_sig = sum(p_adj < 0.05 & term != "(Intercept)",
    pseudoR2 = max(pseudoR2, na.rm = TRUE) # Model-level metric
  ) %>%
  arrange(desc(pseudoR2))
# So n_sig = # of predictors which survive FDR at 0.05 significance (i.e., # of
  ↪ covariates associated with missingness)

# Predictors of missingness for missing vars in probit (dmean_num_ENROLLMENT_COST and
  ↪ DIST_FROM_NEAREST_PRIMARY_CLASS)

sel_predlogit_NA_all <- logit_missing_results_all %>%
  filter(
    missing_var %in% miss_vars,
    term != "(Intercept)",
    p_adj < 0.05) %>%
  mutate(predictor = term) %>%
  select(missing_var, predictor, estimate, p_adj) %>%
  arrange(missing_var, p_adj, desc(abs(estimate)))

# Plot results: variables with most non-random missingness

plt_sel_logit_NA_all <- logit_missing_results_all %>%
  group_by(missing_var) %>%
  summarise(pseudoR2 = max(pseudoR2, na.rm = TRUE)) %>%
  ggplot(aes(reorder(missing_var, pseudoR2), pseudoR2)) +

```

```

# "'remove' and 'certainty' options give the same result"
# "but 'certainty' is intended for situations where there is only one PSU in the
  ↳ population stratum, which is sampled with certainty (also called 'self-representing'
  ↳ PSUs or strata). With 'certainty' no warning is generated for strata with only one
  ↳ PSU. Ordinarily, svydesign will detect certainty PSUs, making this option
  ↳ unnecessary."

# Will thus use "average" for handling single-PSU strata (e.g., "Jhelam Valley") when
  ↳ estimating variance
options(survey.lonely.psu = "average")

# Create weighted survey design model
selection_design <- svydesign(

  ids = ~FSU_SL_NO + HHID,

  # `?svydesign`: "For multistage sampling the id argument should specify a formula with
    ↳ the cluster identifiers at each stage."

  # Sampling occurred in two stages: the first-stage units (FSU) were rural
    ↳ villages/urban blocks (FSU_SL_NO), and the ultimate stage units (USUs) were
    ↳ households (HHID)

  strata = ~interaction(
    as.character(STATE),
    as.character(STRATUM),
    as.character(SUB_STRATUM_NO),
    drop = TRUE # Nothing dropped yet
  ),

  # Sampling stratified into state, stratum (district), and substratum (rural/urban, with
    ↳ multiple urban substrata if population dense enough)

  weights = ~weight,
  data = selection_df,

```

```

    nest = TRUE # Clusters (PSUs) are nested within strata (states*strata*substrata)
)

# Estimate weighted model
model_probit_selection <- svyglm(
  f_probit,
  design = selection_design,
  family = binomial(link = "probit")
) # Rows with an NA for a probit-relevant variable dropped!

# Derive IMR
# For now, calculate IMR for all rows. Don't drop rows with NAs in unrelated columns:

# Rows with no NA under probit-relevant variable
needed_vars <- all.vars(f_probit) # = c(probit_vars, "enrolled")

# Take only complete cases of variables in f_probit
comp_cases <- complete.cases(selection_df[, needed_vars, drop = FALSE])

# Predict only on complete rows; NA for rest
lp <- rep(NA_real_, nrow(selection_df)) # full-length placeholders
lp[comp_cases] <- predict(model_probit_selection,
  newdata = selection_df[comp_cases, , drop=FALSE],
  type = "link") # to return linear predictors

# Build IMR
phi_lp <- dnorm(lp)
Phi_lp <- pnorm(lp)

```

```

# Remaining NAs could be from dividing by 0.
# Ensure against this by keeping Phi_lp \in (0,1) strictly
Phi_lp <- pmin(pmax(Phi_lp, .Machine$double.eps), 1 - .Machine$double.eps)
# .Machine$double.eps = 2.220446e-16 on my device

selection_df <- selection_df %>%

  mutate(
    IMR = case_when(
      enrolled == "Yes" ~ phi_lp / Phi_lp,
      enrolled == "No" ~ -phi_lp / (1 - Phi_lp),
      TRUE ~ NA_real_ # NA if missing predictor
    )
  )

```

Chunk 10: Participation model: Derive AME

```

# Use automatic differentiation (AD) for SE delta method gradients
autodiff(autodiff = TRUE, install = FALSE) # install=TRUE if self-managing Python install
options(marginalEffects_parallel = FALSE) # Make sure it's off
sel_data <- selection_df[comp_cases, , drop = FALSE]

num_samp <- 200
set.seed(999)
final_draft <- TRUE # Set TRUE when rendering final draft
newdata_ame <- if (final_draft) sel_data else sel_data %>% slice_sample(n = num_samp)

# start.time <- Sys.time()
mfx_all <- avg_slopes(
  model_probit_selection,
  newdata = newdata_ame,
  wts = "weight",

```

```

# # More manual tests
# stringdist("East Godavari", "Godavari East", method = "qgram")
# # 0
# stringdist("East Godavari", "Godavari East", method = "jaccard")
# # 0

evaluate_distances <- function(pairs,
                               methods, thresholds,
                               col1 = "str1", col2 = "str2") {
  if(length(methods) != length(thresholds)) {
    stop("\"methods\" and \"thresholds\" must have the same length.")
  }
  # Ensure chr type
  pairs <- pairs %>%
    mutate(
      !!col1 := as.character(.data[[col1]]),
      !!col2 := as.character(.data[[col2]])
    )

  # Compute distances for each method
  out <- bind_rows(
    lapply(seq_along(methods), function(i) {
      m <- methods[i]
      th <- thresholds[i]

      pairs %>%
        transmute(
          str1 = .data[[col1]],
          str2 = .data[[col2]],
          method = m,
          distance = stringdist(str1, str2, method = m),

```

```

    threshold = th,
    match = distance <= th
  )
})
)

# Keep a consistent ordering
out %>% arrange(str1, str2, method)
}

# Some sample pairs
pairs <- tribble(
  ~str1, ~str2,
  "Baleshwar", "Balasore",
  "Jammu & Kashmir", "Jammu and Kashmir",
  "East Godavari", "Godavari East",
  "Sikim", "Sikkim",
  "Mumbai", "Mumbai",
  "24-Parganas ( North )", "North Twenty Four Parganas",
  "North Twenty Four Pargan*", "North Twenty Four Parganas",
  "Sahibzada Ajit Singh Nag*", "Sahibzada Ajit Singh Nagar",
  "Sri Potti Sriramulu Nell*", "Sri Potti Sriramulu Nellore"
)

#### Helper vectors ####
methods <- c("soundex", "qgram", "jw", "dl", "osa")
thresholds <- c(0, 0, 0.15, 2, 1)

# soundex=0 --> Phonetic variants in anglicization allowed (if coarsely measured)
# qgram = 0 --> Some rearrangements of characters allowed
# jw<=0.15 --> Respellings + vowel swaps with 0.85 similarity allowed
# dl<=2 --> <=2 insertions + deletions + substitutions + transpositions allowed

```

```

# osa<=1 --> 1 typo allowed
# Template for testing these vs. other methods/thresholds can be found at start of chunk

# Evaluate distances and test joining methods
# colSums(is.na(joined_df))
# # 53 in 01, 112 in 07-08, 89 in 17-18
# sum(apply(joined_df, 1, anyNA))
# # 166/734
# evaluate_distances(pairs, methods, thresholds)

```

Chunk 17: Match districts: Define joining functions

```

# A. Loop over the methods*thresholds
# B. For each methods*thresholds pair, fuzzy full join all still-unmatched rows
# C. Return joined, unmatched_df1, unmatched_df2
fuzzy_join_sequence <- function(df1, df2,
                                dist1, state1,
                                dist2, state2,
                                methods, thresholds, mode = "full") {
  # Attach row IDs to track unmatched
  df1_id <- df1 %>% mutate(.id1 = row_number())
  df2_id <- df2 %>% mutate(.id2 = row_number())

  df1_curr <- df1_id
  df2_curr <- df2_id
  matched_all <- tibble()

  for(i in seq_along(methods)) {
    full_j <- stringdist_join(
      df1_curr, df2_curr,

```

```

    by = setNames(c(dist2, state2), c(dist1, state1)),
    mode = mode,
    method = methods[i],
    max_dist = thresholds[i],
    distance_col = "dist"
  )

  # Keep only pairs (i.e., those with both sides present) and rank them
  matched_i <- full_j %>%
    filter(!is.na(.id1) & !is.na(.id2)) %>%
    group_by(.id1) %>%
    slice_min(order_by = dist, n = 1, with_ties = FALSE) %>%
    ungroup() %>%
    group_by(.id2) %>%
    slice_min(order_by = dist, n = 1, with_ties = FALSE) %>%
    ungroup()
  matched_all <- bind_rows(matched_all, matched_i)

  # Remove them from the to-do sets
  df1_curr <- df1_curr %>% filter(!(.id1 %in% matched_i$.id1))
  df2_curr <- df2_curr %>% filter(!(.id2 %in% matched_i$.id2))
}

list(
  joined = matched_all,
  unmatched_df1 = df1_curr %>% select(-.id1),
  unmatched_df2 = df2_curr %>% select(-.id2)
)
}

# Default is full join so that this can be used for both matching and diagnostics

```



```

# For each df name in input vector:
# A. Extract its "01"/"05"/... suffix
# B. Find the suffix's district_/state_ columns in joined_df.
# C. Run the fuzzy join sequence on each column. (Recall justification of fuzzy joining
  ↳ states in Chunk 6: Match districts: Construct district tracking dfs.)
# D. Repeat with the predecessor suffix.
merge_dfs_into_tracker <- function(
  dfs,
  tracker,
  years_of_interest,
  methods, thresholds,
  flag = FALSE
) {
  # 1. Initialize both joined_df and (if flag=TRUE) flagged_df
  joined_df <- tracker
  if (flag) {
    flagged_df <- tracker
    # Prepare placeholder vector for each df-name
    matched_lists <- lapply(names(dfs), function(x) rep(NA_character_, nrow(tracker)))
    names(matched_lists) <- names(dfs)
  }

  year_suffixes <- substr(as.character(years_of_interest), 3, 4)
  unmatched_list <- vector("list", length(dfs))
  names(unmatched_list) <- names(dfs)

  # 2. Loop over each auxiliary data-frame name
  for (d_name in names(dfs)) {
    df_d <- dfs[[d_name]]
    suffix <- stringr::str_extract(d_name, "\\d{2}$")
    stopifnot(!is.na(suffix))
  }
}

```

```

# Find exactly one district_XX / state_XX in df_d
dist_d <- grep("^district_", names(df_d), value=TRUE) %>%
  grep("code", ., invert=TRUE, value=TRUE)
state_d <- grep("^state_", names(df_d), value=TRUE) %>%
  grep("code", ., invert=TRUE, value=TRUE)
stopifnot(length(dist_d)==1, length(state_d)==1)

df_unmatched <- df_d
if (flag) mv <- matched_lists[[d_name]]

# suffix_chain sorted by absolute distance from said suffix
year_nums <- as.integer(year_suffixes)
suffix_num <- as.integer(suffix)
order_idx <- order(abs(year_nums - suffix_num))
suffix_chain <- year_suffixes[order_idx]

# Work through suffixes, closest to furthest
for (suf in suffix_chain) {
  dist_cols_j <- grep(paste0("^district_", suf, "(?:$|_)"),
    names(joined_df), value=TRUE)
  state_cols_j <- grep(paste0("^state_", suf, "(?:$|_)"),
    names(joined_df), value=TRUE)

  for (dist_col_j in dist_cols_j) {
    ending <- sub(paste0("^district_", suf), "", dist_col_j)
    state_col_j <- paste0("state_", suf, ending)
    if (!state_col_j %in% state_cols_j) next
  }

  # 2a. Multi-threshold fuzzy join
  res <- fuzzy_join_sequence(
    df_unmatched, joined_df,
    dist1 = dist_d,

```

```

state1 = state_d,
dist2 = dist_col_j,
state2 = state_col_j,
methods = methods,
thresholds = thresholds,
mode = "full"
)

# 2b. Pull other cols into joined_df
other_cols <- setdiff(names(df_d), c(dist_d, state_d))
if (nrow(res$joined)>0 && length(other_cols)>0) {
  extra <- res$joined %>%
    select(.id2, all_of(intersect(other_cols, names(res$joined)))) %>%
    arrange(dist, .id2) %>%
    group_by(.id2) %>%
    slice_min(order_by = dist, n = 1, with_ties = FALSE) %>%
    ungroup()

  # Merge into joined_df
  joined_df <- joined_df %>%
    mutate(.row = row_number()) %>%
    left_join(extra, by = c(".row"=".id2")) %>%
    select(-.row)

  # If flag, merge into flagged_df
  if (flag) {
    flagged_df <- flagged_df %>%
      mutate(.row = row_number()) %>%
      left_join(extra, by = c(".row"=".id2")) %>%
      select(-.row)
  }
}

```

```

# 2c. If flag=TRUE, record which district got matched this iteration
if (flag && nrow(res$joined)>0) {
  col_match <- grep(
    paste0("^", dist_d, "(?:$|\\.x$)"),
    names(res$joined), value=TRUE
  )[1]
  info <- res$joined[, c(".id2", col_match), drop=FALSE]
  colnames(info)[2] <- "match_dist"
  info <- info %>%
    group_by(.id2) %>%
    slice_head(n = 1) %>%
    ungroup()

  idx <- info$.id2
  new_vals <- info$match_dist
  blank <- is.na(mv[idx])
  mv[idx[blank]] <- new_vals[blank]
}

# 2d. Update the df of unmatched
df_unmatched <- res$unmatched_df1
if (nrow(df_unmatched)==0) break
}

if (nrow(df_unmatched)==0) break
}

# 2e. Once done, append flag-column to flagged_df
if (flag) {
  flagged_df[[d_name]] <- mv
}
unmatched_list[[d_name]] <- df_unmatched

```

```

    if (nrow(df_unmatched)>0) {
      warning(sprintf("Some rows in %s were never matched", d_name))
    }
  }

  # 3. Finalize flagged_df if flag=TRUE
  if (flag) {
    max_suf <- year_suffixes[which.max(as.integer(year_suffixes))]
    keep_cols <- names(flagged_df)[
      (grepl("^district_", names(flagged_df)) & !grepl("code", names(flagged_df))) |
      names(flagged_df) %in% names(dfs) |
      names(flagged_df) == paste0("state_", max_suf)
    ]
    flagged_df <- flagged_df[, keep_cols, drop=FALSE]
  }

  # 4. Return!!!
  if (flag) {
    list(
      joined_df = joined_df,
      unmatched = unmatched_list,
      flagged_df = flagged_df
    )
  } else {
    list(
      joined_df = joined_df,
      unmatched = unmatched_list
    )
  }
}

```

```

# rook_neighbors <- joined_geom %>% st_rook()
# rook_neighbors %>% lengths() %>% mean()
# # 4.780165 average neighbors per district
# queen_neighbors <- joined_geom %>% st_queen()
# queen_neighbors %>% lengths() %>% mean()
# # 4.783471
# # Average neighbor counts are nearly equivalent

# # Test for fastest contiguity measure
# start.time <- Sys.time()
nb_2020 <- joined_df %>%
  poly2nb(queen = FALSE)
# end.time <- Sys.time()
# end.time - start.time
# # With queen = TRUE: 32.64976 secs
# # With queen = FALSE: 31.15051 secs
# # Rook contiguity slightly faster

# Binary adjacency matrix. Rook-contiguity neighbor list used for speed, conceptual
  ↪ interpretability
W_2020 <- nb2mat(nb_2020, style = "B", zero.policy = TRUE)
# W_2020[i,j] == 1 if districts i and j share a border, 0 else

```

Chunk 25: 2SLS: Controls and 2SLS formula

```

#### Control vectors ####
controls_needlag <- c("consumption_0708", "gini_cons_0708")
controls_nolag <- c(
  "pct_urban", "avg_hh_size", "dependency_ratio",
  "pct_fem_head",
  "pct_hindu", "pct_muslim",

```

```

    "pct_st", "pct_sc", "pct_obc",
    "pct_small_land", "pct_medium_land", "pct_large_land",
    "pct_head_lit_to_primary",
    "pct_head_secondary_plus")

# Extract spatial lags
controls_lagged <- paste0("W_", controls_needlag)

#### IV Formula ####
make_iv_formula <- function(dep, endog, exog, inst) {
  as.formula(paste0(
    dep, " ~ ",
    paste(c(endog, exog), collapse = " + "),
    " | ",
    paste(c(inst, exog), collapse = " + ")
  ))
}

```

Chunk 26: 2SLS: Baseline 2SLS models

```

# Formula syntax: outcome ~ endogenous + controls | instrument + controls
model_consumption_iv <- ivreg(
  make_iv_formula(
    dep = "consumption_pct_change",
    endog = "EMIE",
    exog = c(controls_needlag, controls_nolag),
    inst = "wavg_ling_degrees"
  ),
  data = joined_df
)

```

```

# First-stage regression
first_stage_consumption <- lm(
  reformulate(
    c("wavg_ling_degrees", controls_needlag, controls_nolag),
    response = "EMIE"
  ),
  data = joined_df # Complete cases already enforced, so ivreg() and lm() will use the
  ↪ same estimation sample
)

# Cluster SEs by state
vcov_first_stage_consumption <- vcovCL(first_stage_consumption, cluster = ~ state_20)
vcov_model_consumption_iv <- vcovCL(model_consumption_iv, cluster = ~ state_20)

# Repeat for Gini model
model_gini_iv <- ivreg(
  make_iv_formula(
    dep = "gini_change",
    endog = "EMIE",
    exog = c(controls_needlag, controls_nolag),
    inst = "wavg_ling_degrees"
  ),
  data = joined_df
)

vcov_model_gini_iv <- vcovCL(model_gini_iv, cluster = ~ state_20)

```



```

    comma(round(.x, 0)),
    # Otherwise, keep two decimals
    sprintf("%.2f", .x)
  )
)
)# %>% mutate(across(Min:SD, ~ signif(.x, 3))) # For 3 significant digits

```

Chunk 28: Multicollinearity check

```

X <- c("EMIE", controls_needlag, controls_nolag) %>%
  reformulate() %>%
  model.matrix(data = joined_df)

# # Number of independent columns:
# qr(X)$rank
# # Num. total columns:
# ncol(X)
# # Recall: If qr(X)$rank < ncol(X), exact collinearity.

# Heuristic: kappa>30 implies severe multicollinearity
X_no_intercept <- X[, colnames(X) != "(Intercept)", drop = FALSE]
keep <- apply(X_no_intercept, 2, sd, na.rm = TRUE) > 0
X_scaled <- scale(X_no_intercept[, keep, drop = FALSE])
kappa_val <- kappa(X_scaled, exact = TRUE)

# # Check for linear dependence
# X_df <- as.data.frame(X_scaled)
# alias(lm(
#   consumption_pct_change ~ .,
#   data = cbind(consumption_pct_change = joined_df$consumption_pct_change, X_df)
# ))

```

```

# # Heuristic: VIF > 10 implies problematic collinearity
# vif(lm(
#   reformulate(
#     c("EMIE", controls_needlag, controls_nolag),
#     response = "consumption_pct_change"
#   ),
#   data = joined_df
# ))
#
# vif(model_consumption_iv)
# vif(model_gini_iv)

```

Chunk 29: Geospatial: Spatial autocorrelation tests

```

# Build row-standardized spatial weights (listw) object
# Recall: Number+order of rows in 2SLS models = number+order used for spatial weights
listw_2020 <- nb2listw(nb_2020, style = "W", zero.policy = TRUE) # TRUE for, say, islands

#Extract residuals from IV models
resid_cons <- residuals(model_consumption_iv)
resid_gini <- residuals(model_gini_iv)
resid_fscons <- residuals(first_stage_consumption)

# Global Moran's I on residuals
m_cons_resid <- moran.test(resid_cons, listw_2020, zero.policy = TRUE)
m_gini_resid <- moran.test(resid_gini, listw_2020, zero.policy = TRUE)
m_fscons_resid <- moran.test(resid_fscons, listw_2020, zero.policy = TRUE)

# # p-val's via Monte Carlo (moran.test() assumes asymptotic normality)
# set.seed(999)

```